SQL PROJECT : **EMPLOYEE AND DEPARTMENT MANAGEMENT SYSTEM**

1.Project Description:

This project manages employee information and the departments.Using SQL queries, we can analyze employee distribution, department information, and employee statatics. Managing employee information efficiently is important for maintaining proper records, analyzing workforce distribution, and understanding departmental structures. This project simulates a simplified version of such a system by creating a database that stores employee details and department information.

In this project, two main tables are created: the **Employees table** and the **Departments table**. The Employees table stores information such as employee ID, employee name, age, and the department to which the employee belongs. The Departments table stores the department ID and the department name.

To maintain data integrity and relationships between tables, the database uses **Primary Keys** and **Foreign Keys**. The department_id in the Departments table acts as the **Primary Key**, uniquely identifying each department. The same column appears in the Employees table as a **Foreign Key**, establishing a relationship between employees and their respective departments.

Main Objectives :

- Store employee and department information
- Identify which employees belong to which department
- perform analysis using SQL queries
- Use joins to connect tables
- Use aggregate functions

SQL Query Used:

- SELECT
- WHERE
- ORDER BY
- GROUP BY
- HAVING
- JOIN
- SUBQUERY
- CASE
- VIEW
- CTE

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

join* exp3* practice* case* march_demo* SQL File 7* EMPLOYEE AND DEPARTMEN...

Limit to 1000 rows

```sql
create database Sql_project;
use Sql_project;

CREATE TABLE Departments (
department_id INT PRIMARY KEY,
department_name VARCHAR(50) NOT NULL
);


CREATE TABLE Employees (
emp_id INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
age INT,
department_id INT,
FOREIGN KEY (department_id) REFERENCES Departments(department_id)
);

INSERT INTO Departments VALUES
(101,'IT'),
(102,'Human Resources'),
(103,'Finance'),
(104,'Marketing');


INSERT INTO Employees VALUES
(1,'Rahul',28,101),
(2,'Anita',32,102),
(3,'John',26,101),
(4,'Meera',30,103),
(5,'David',35,102),
(6,'Sara',29,104);

select * from Departments;
```

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

join* exp3* practice* case* march_demo* SQL File 7* EMPLOYEE AND DEPARTMEN...

Limit to 1000 rows

```sql
select * from Employees;

-- SQL QUERIES

-- show all employees and departments


select * from Departments;
select * from Employees;


-- 2.show name of employee

select name from Employees;

-- 3.employee older than 30
select name,age from Employees where age>30;


-- 4 employee btw age 25 and 30
select name,age from Employees where age between 25 and 30;

-- 5 employee from department id
select * from Employees where department_id=101;


-- 6 sort employee by age
select * from Employees order by age;

-- 7 descending order sort
select * from Employees order by emp_id desc;

-- 8 count function
```

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

join* exp3* practice* case* march_demo* SQL File 7* EMPLOYEE AND DEPARTMEN...

Limit to 1000 rows

```
-- 8 count function
select count(*) from Employees;
select count(age) from Employees;


-- 9 distinct values on employees
select distinct(age) from Employees;

-- 10 average
select avg(age) from Employees;

-- 11 max
select max(age) from Employees;

-- 12 min
select min(age) from Employees;


-- 13 join show employee with department name

select e.name,d.department_name
from Employees  e join Departments d
on e.department_id= d.department_id;

-- 14 show employee age,name and department

select e.name,e.age,d.department_name from
Employees  e join Departments d
on e.department_id=d.department_id;

-- 15 show employee in human resources
select e.name from
Employees  e join Departments d
```

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

join* exp3* practice* case* march_demo* SQL File 7* EMPLOYEE AND DEPARTMEN...

Limit to 1000 rows

```
-- 15 show employee in human resources
select e.name from
Employees  e join Departments d
on e.department_id=d.department_id
where d.department_name="Human Resources";

-- 16 show employee in it departments;
select e.name from
Employees  e join Departments d
on e.department_id=d.department_id
where d.department_name="IT";

-- 17 Count employees in each department
select department_id,count(*) as total from Employees group by department_id;

-- 18 average age by department

select department_id,round(avg(age)) from Employees group by department_id;

-- 19 Departments with more than one employee

select department_id,count(*) from Employees group by department_id
having count(*) >1;

-- 20 Employee with maximum age
 Select name
from Employees
where age = (select max(age) from Employees);


-- 21 Employees older than average age

select name
```

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

join* exp3* practice* case* march_demo* SQL File 7* EMPLOYEE AND DEPARTMEN...

Limit to 1000 rows

```sql
-- 21 Employees older than average age

select name
from Employees
where age > (select avg(age) from Employees);

-- 22 Employees in same department as Rahul
select name
from Employees
where department_id =
(
select department_id
from Employees
where name = 'Rahul'
);

-- 23 categorize employees by age

use Sql_project;

select name,age,
CASE
    when age<28 then "young employee"
    when age between 28 and 32 then "mid age employee"
    else "senior employee"
end as Age_category
from Employees;



-- create a view

create view emplovee dept as
```

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

join* exp3* practice* case* march_demo* SQL File 7* EMPLOYEE AND DEPARTMEN...

Limit to 1000 rows

```sql
select name,age,
CASE
    when age<28 then "young employee"
    when age between 28 and 32 then "mid age employee"
    else "senior employee"
end as Age_category
from Employees;



-- create a view

create view employee_dept as
select e.emp_id,e.name,e.age,d.department_name
from Employees e join Departments d
on e.department_id=d.department_id;


select * from employee_dept;

-- cte employee older than 30

with older_emp as
(
select * from Employees
where age>30
)
select * from older_emp
```